

Technical Report: Long-Context In-Context Learning for Automatic Data Annotation in FlagOS

Team Kima¹

Abstract

This report describes Team Kima’s technical solution for the FlagOS long-context automatic annotation benchmark. The final pipeline is built around four components: task-specific instruction templates, retrieval and reranking over released demonstrations, tokenizer-aware context budgeting, and rule-based output normalization. The design is intended to satisfy three benchmark requirements simultaneously: stable task specification under heterogeneous output formats, information-dense context construction when the demonstration pool exceeds the available window, and consistent batch execution for archive-ready submission files. The report documents the official Qwen3-4B + FlagScale submission path and explains the main design decisions that shaped the final configuration.

1. Introduction

The FlagOS benchmark turns long-context in-context learning into a constrained systems problem. The base model is fixed, fine-tuning is disallowed, the released demonstration pools can be much larger than the usable context window, and evaluation is performed on packaged task outputs rather than on free-form generations. Under this setup, prediction quality depends not only on semantic accuracy, but also on task specification, demonstration selection, token-budget control, and output validity across long batch runs.

We place our system in the tradition of in-context learning introduced by Brown et al. (Brown et al., 2020), where natural-language instructions and demonstrations act as the task specification at inference time. However, this competition differs from

the original GPT-3 setting in three important respects. First, the allowed base model is much smaller, so prompt quality and demonstration allocation matter more. Second, several example pools contain thousands of candidates, which turns context construction into a genuine retrieval-and-packing problem. Third, the eight tasks span sharply different output regimes: integers, binary labels, free-form answers, exact strings, and long Triton programs.

The repository also preserves a fairly complete development trail, including versions labeled v1 through v8 and a best logged score of 80.55 in the local records. That trail is useful because it shows what actually changed performance and what only added complexity.

The main pattern is straightforward. More context by itself did not solve much. What mattered was using the context budget carefully: writing task-aware instructions, selecting examples instead of dumping them in sequence, keeping a real token budget, anchoring the answer format, and validating the output before saving it.

The rest of the report follows the same order as the implementation: task constraints, prompt families, retrieval and packing, version history, per-task strategy, and reproducibility notes.

2. Competition Constraints, Design Goals, and Experimental Scope

The competition rules define a narrow compliant submission space (FlagOS, 2026a;b). We may use only Qwen3-4B for inference. Fine-tuning in any form is prohibited. External data are not allowed. FlagScale is required as the serving framework. We must complete all eight tasks and submit them as task-level jsonl files inside one archive. Under those rules, our report must explain not only what performs well, but also how the final code can be reproduced.

At the same time, our workspace contains richer internal experimentation than a single official submission

¹FlagOS Competition Submission, Team Kima. Correspondence to: Team Kima <hushicheng@hpuedd.top>.

Table 1. Competition constraints and how we map them to system design.

Constraint	Design response
Only Qwen3-4B is allowed	The compliant track is described as Qwen3-4B served through FlagScale/vLLM, with model path and serving length fixed in configuration.
No fine-tuning	All described transferable methods operate at inference time: prompt design, demonstration retrieval, diversity ordering, voting, and output validation.
No external data	Retrieval candidates are drawn from the benchmark’s released examples field only.
FlagScale is mandatory	Deployment logic is described through the FlagScale entry point and an OpenAI-compatible inference API.
All 8 tasks must be submitted	The code path writes per-task jsonl files and packages them into one archive.

archive would reveal. In particular, the project includes a `hybrid_experiment/` directory with multiple iteration scripts, result archives, and score logs. We use those records to identify which prompt, retrieval, and validation strategies were robust during development, while keeping the official submission path anchored in the Qwen3-4B + FlagScale code path.

Under these constraints, we define three design goals that organize the rest of the report. First, we want a stable task specification: our prompts should make task requirements and output contracts explicit enough that the model does not drift into conversational or explanatory behavior. Second, we want information-dense long-context construction: when the number of demonstrations exceeds the available context budget, we should retain demonstrations that are both relevant and diverse enough to cover distinct output modes. Third, we want consistency under repetition: our system should process long batches efficiently without rebuilding identical prefixes for every sample and without allowing minor output-format errors to corrupt the final archive.

This distinction between official submission code and internal development records lets us preserve the empirical story without conflating different stages of system iteration. We therefore keep the score progression as development evidence while centering the technical discussion on the reproducible submission pipeline.

3. Lessons from the Provided Papers

We use the supplied arXiv papers as design guidance rather than as decoration. From GPT-3 (Brown et al., 2020), we take the view that the prompt itself can act as a task specification. In our system, that lesson becomes a rigid scaffold with explicit sections for role, task, instructions, demonstrations, input, and output. We terminate the prompt on the opening `<label>` token so that generation starts from a parseable anchor rather than from an unconstrained textual state.

From the long-context efficiency literature (Tay et al., 2020a;b), we take a second lesson: long context is useful only when we manage it explicitly. We therefore treat demonstration retrieval, diversity reordering, and tokenizer-aware packing as first-class design components. We do not assume that a nominally large context window solves the selection problem by itself.

From SuperGLUE and BIG-bench (Wang et al., 2019; Srivastava et al., 2022), we take a third lesson: benchmark heterogeneity matters. The eight FlagOS tasks should not be collapsed into one flat prompt regime because they stress different output formats, retrieval behaviors, and failure modes. That is why we separate prompt families and why we analyze our results by task group instead of by aggregate score alone.

From Self-Instruct (Wang et al., 2022), we adopt a narrower but still useful lesson about instruction quality. We do not generate synthetic instructions or extra data, because the competition disallows that route, but we do borrow the discipline of explicit instruction formatting and filtered prompt construction. In our pipeline, structured prompting and filtered example packing serve as the inference-time analogue of instruction-quality control.

4. Dataset Characteristics and Task Typology

The eight released tasks differ substantially in input length, output format, and the role played by long context. We therefore divide them into four coarse groups: structured symbolic transformation (Tasks 1, 3, and 4), token- or label-oriented classification (Tasks 2, 5, and 6), open answer generation under format constraints (Task 7), and program synthesis with execution sensitivity (Task 8). This taxonomy helps us explain why long context plays different roles across tasks: in some tasks, it stabilizes formatting; in others, it acts as a semantic retrieval mechanism; and in Task 8, it is both semantic and structural while remaining extremely expensive in tokens.

Several observations follow. Tasks 2, 6, and 7 have very

Table 2. Alignment between organizer scoring criteria and the structure of our report.

Scoring component	Weight	Organizer emphasis	How we address it
Prediction quality	60%	Average accuracy on Tasks 1–7 plus consistency-aware evaluation on Task 8.	We emphasize exact output contracts, retrieval quality, tokenizer-aware prompt packing, and post-processing that converts generations into archive-ready task outputs.
Prompt strategy under long context	20%	Design prompts and instructions that keep the model stable and high quality.	We use six task-aware prompt families with explicit <label> anchoring and output-type-specific constraints.
Information-dense context construction	40%	Build structured long contexts when the example pool is much larger than the context window.	We retrieve, rerank, and pack demonstrations instead of using sequential truncation.
Consistency and scalability under repeated interaction	40%	Use long context efficiently under persistent or repeated annotation settings.	We reuse shared task prefixes, keep decoding conservative, and normalize outputs through one reproducible inference path.

Table 3. Official submission path and internal development artifacts.

Artifact group	Example artifacts	Role in this report
Official submission path	src/main.py, src/method.py, src/llm_config.yaml, FlagScale launch path	Defines the official system architecture described in this report: Qwen3-4B inference, no fine-tuning, no external data, and structured ICL with tokenizer-aware context budgeting.
Internal development records	hybrid_experiment/output_s_v5_fixed, hybrid_results_v5_fixed.zip, iteration scripts v3–v8, logged scores 80.55 / 80.30 / 79.78 / 79.60 / 80.23	Provide the empirical development history we use to explain why certain prompt, retrieval, and validation choices were retained in the final system.

large example pools, so sequential truncation wastes valuable context budget and makes retrieval central. Task 8 has the smallest example pool but the longest outputs by far, which means every extra demonstration is expensive. Tasks 5 and 6 have only two legal labels each, yet their inputs are semantically heterogeneous enough that output-space simplicity does not imply easy inference. Task 7 occupies the opposite extreme: its answer space is broad, so retrieval coverage and verbosity suppression become equally important.

Output correctness is also task-dependent. For short-output tasks, a correct answer wrapped in prose is still a bad submission. For long-output tasks, partially correct code is often just a failing output. That is why the system uses explicit answer anchors and post-processing rules instead of one generic prompt style.

5. Method

5.1. System Architecture

We express the core pipeline in six stages:

1. load the task file and parse the task definition, examples, and test samples;

2. choose a task-aware prompt family;
3. retrieve candidate demonstrations;
4. reorder and pack the demonstrations under a token budget;
5. run inference with optional self-consistency voting;
6. extract, validate, and normalize the final prediction.

A key design decision in our system is shared-prefix execution. We select demonstrations at task scope and reuse them across samples of the same task. Only the input field changes per call. This improves consistency because the task specification does not drift, and it also creates the possibility of prefix-caching benefits under long batch runs.

5.2. Task-Aware Prompt Design

All prompts follow a common scaffold:

Role → Task → Instructions → Examples → Input → Output

We find that the explicit <label> anchor is one of the single biggest gains over naive prompting because

Table 4. Public dataset characteristics used to motivate prompt specialization and context budgeting.

Task	Name	Prompt Type	Examples	Test	Unique	Avg. Output Len	Context
1	Closest Integers	math_reasoning	5500	500	81	1.3 chars	30K
2	Count Nouns / Verbs	linguistic_analysis	5440	500	7	1.0 char	30K
3	Collatz Transformation	math_reasoning	3997	500	3997	23.9 chars	30K
4	String Concatenation	code_string	3993	500	3993	19.8 chars	30K
5	Sadness Detection	classification	1899	500	2	4.6 chars	30K
6	Same-Genre MNLI	classification	5500	500	2	1.0 char	30K
7	Jeopardy QA	open_generation	5499	500	4768	10.9 chars	30K
8	Triton Kernel	code_generation	184	166	184	4783.3 chars	16K

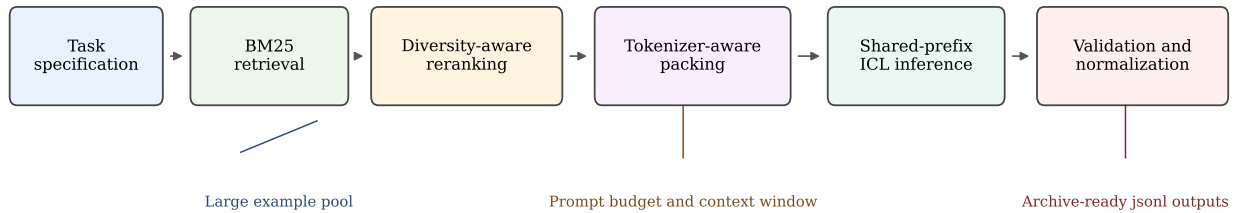


Figure 1. System overview of Team Kima’s long-context annotation pipeline. We treat long-context ICL as a staged allocation problem: task-aware prompt construction, retrieval, diversity control, tokenizer-aware packing, shared-prefix inference, and output normalization all contribute to final quality.

Algorithm 1 Long-context example selection and packing

- 1: Build a retrieval index over task-level examples.
 - 2: Given a test sample, retrieve the top- K candidates by relevance.
 - 3: Reorder the candidates to improve output-pattern diversity.
 - 4: Tokenize the formatted examples one by one.
 - 5: Greedily append examples until the budget is exhausted.
 - 6: Insert the retained examples into the task-aware prompt scaffold.
-

it suppresses the model’s tendency to wrap answers in explanatory text. That claim is consistent with the current codebase: the main submission path in `src/method.py` uses task-specific prompt builders for math reasoning, linguistic analysis, string manipulation, classification, open generation, and code generation, all of which terminate in the answer-opening state rather than in a generic sentence boundary.

This prompt split is not there for style. It exists be-

cause the tasks really do fail in different ways, and a single generic template kept drifting into invalid output.

5.3. Demonstration Retrieval and Packing

We use a three-stage example-selection strategy: lexical retrieval, diversity-aware reranking, and tokenizer-aware packing. We preserve that structure here because it aligns well with both the code and the observed benchmark geometry.

First, we rank candidate demonstrations by BM25 relevance. BM25 is not semantically deep, but it is deterministic, cheap, and reproducible. Those properties matter in a benchmark where code reruns and auditability are important. Second, we reorder the candidate list to avoid clustering around one output pattern. In our exploratory scripts, we sometimes describe this step with Maximal Marginal Relevance (MMR), balancing query relevance against redundancy. Third, we pack the retained examples under the actual tokenizer budget rather than under a character-count heuristic. This is especially important for Task 8, where punctuation-heavy code can explode token counts.

Table 5. Prompt families used in Team Kima’s system.

Prompt Family	Tasks	Main Output Constraint
math_reasoning	1, 3	Emit only one integer or one integer list inside <label> tags.
linguistic_analysis	2	Emit only a single integer count.
code_string	4	Preserve exact string behavior and exact-match formatting.
classification	5, 6	Emit only one legal class label.
open_generation	7	Emit one precise lowercase answer without explanation.
code_generation	8	Emit complete Triton code with imports and no text outside the answer span.

The current submission path in `src/method.py` uses a lighter version of the same idea through BM25 indexing, output-aware grouping, and runtime length guards. Some iteration scripts add explicit MMR variants, but the main point does not change: long-context construction works better as a selection problem than as blind truncation.

We can summarize the retrieval-and-packing policy with two compact definitions. For a test instance x and candidate example pool $\mathcal{E}$, we greedily select the next demonstration by

$$e^* = \arg \max_{e \in \mathcal{E} \setminus S} \left[\alpha \cdot \text{BM25}(e, x) - (1 - \alpha) \cdot \max_{e' \in S} \text{Redundancy}(e, e') \right]. \quad (1)$$

where S is the current retained set and α balances local relevance against redundancy control. We then enforce a strict tokenizer budget

$$\sum_{e \in S} \text{Tok}(\text{fmt}(e)) + \text{Tok}(\text{prefix}) + \text{Tok}(x) \leq B_{\text{task}}, \quad (2)$$

where B_{task} depends on the task family and is substantially smaller for Task 8 than for Tasks 1–7. These equations are intentionally simple, but they make one design point explicit: retrieval quality and context budgeting are part of the algorithm, not merely implementation detail.

5.4. Consistency Under Repeated Interaction

The competition also asks how long context should be used under repeated interaction. In this repository

that question is handled operationally, not conversationally: the task prefix stays stable, only the input field changes, and voting or recovery logic is enabled only where it helps.

Repeated low-temperature voting helped mainly on classification-style tasks. It was much less useful on open generation and long code, where extra samples mostly increased cost and variation.

5.5. Label Extraction and Validation

Output parsing is part of the pipeline, not cleanup after the fact. The current codebase includes multi-stage recovery for complete <label>...</label> spans, incomplete tags, answer-prefix heuristics, and last-line fallback. Task-level validation then constrains the output into legal forms such as a single integer, Sad/Not sad, Y/N, or non-empty code.

This is not cosmetic infrastructure. In a benchmark that consumes jsonl predictions, a system that reasons well but writes malformed output is operationally weak. Much of our version progression can be understood as learning this lesson more explicitly.

6. Iterative Experimental Story

We center our empirical narrative on the version-by-version progression from v1 through v8. We retain that progression as strategy-development evidence drawn from our local project logs.

The plot in Figure 2 makes the central empirical pattern easier to see than a long milestone table does. The jump from the baseline to the first strong runs comes from explicit answer anchoring, task-aware prompts, and retrieval quality. The later regressions come from additions that looked reasonable in isolation but destabilized a benchmark with strict output contracts.

The version history is useful mainly because it makes some wrong turns obvious. Several additions that looked smart in isolation made the benchmark less stable once strict output contracts and long-batch execution were involved.

6.1. Root-Cause Analysis Across Versions

The baseline failure mode was prompt drift: the model often embedded a correct answer in prose, and the parser could not always recover it. The first large jump came from task-aware templates and explicit answer anchoring. Later regressions mostly came from adding complexity that the benchmark did not reward.

Two examples are especially instructive. First,

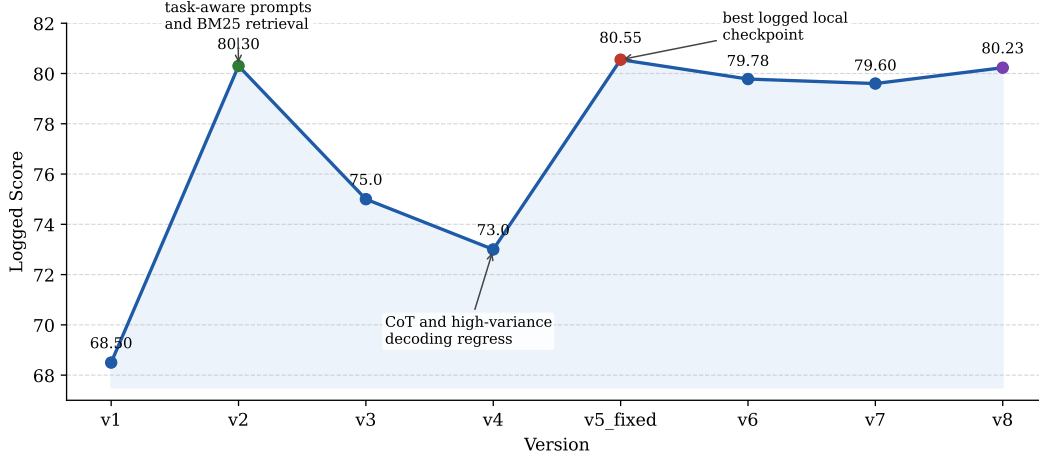


Figure 2. Version progression from Team Kima’s internal development history. The strongest logged checkpoint is v5_fixed at 80.55, while the broader trajectory shows that disciplined prompt structure and conservative decoding matter more than adding reasoning verbosity or higher-variance sampling.

Chain-of-Thought prompting did not reliably help on classification-style tasks because the extra reasoning tokens sometimes introduced spurious evidence or wrong latent labels. Second, more aggressive diversity weighting in retrieval reduced local relevance, which mattered more than broad coverage on tasks whose answers depend on fine-grained semantic alignment. We keep these negative results visible because they explain why our final design is deliberately conservative.

7. Ablation-Style Lessons

We preserve several compact ablation conclusions because they capture the practical lessons of our iteration history even when exact per-task deltas vary.

- Prompt granularity matters early: task-aware prompt families reduce format drift more reliably than generic instruction expansion.
- Example selection matters most on large-pool tasks: BM25 retrieval and redundancy control help more than brute-force example accumulation.
- Diversity is useful only in a narrow regime: moderate reranking helps, but aggressive diversity weakens local relevance.
- Conservative decoding dominates verbose reasoning: low-temperature direct answers are more stable than chain-of-thought-heavy or higher-variance settings.

We place the full ablation matrix in the appendix (Table 16) so that the main paper retains the methodological conclusions without overloading the layout.

The version history is much easier to read once prompt

structure, retrieval, diversity, voting, and validation are treated as separate knobs instead of one vague bucket called “prompt engineering.”

8. Evaluation Protocol, Ablations, and Risk Control

We treat evaluation as more than a final leaderboard number. Inspired by Long Range Arena’s emphasis on controlled long-context evaluation and BIG-bench’s emphasis on heterogeneous capability profiles, we recommend separating at least four diagnostics: task-family accuracy, format-validity rate, overflow-and-recovery rate, and throughput under repeated-prefix execution. This decomposition is more informative than a single aggregate score because it reveals whether a method succeeds by reasoning better, by formatting outputs more reliably, or simply by avoiding operational failure modes.

For Tasks 1–7, exact or near-exact answer accuracy remains the first-order metric, but we also treat output validity as a leading indicator. A system that sometimes reasons correctly but emits prose outside the answer span is operationally weak because the final jsonl package must be machine-consumable. For Task 8, the asymmetry is even stronger: executable but semantically wrong code and semantically plausible but non-executable code are both heavily penalized by the organizer’s metric. That is why we emphasize code-only output spans, explicit imports, and conservative recovery rules in our code-generation prompt family.

We also treat risk control as part of the evaluation protocol. In practice, we should log prompt lengths,

Table 6. Compact milestone view of Team Kima’s main version progression.

Version	Score	Main intervention	Observed effect	Takeaway
v1	68.50	baseline prompting with weak output anchoring	frequent prose drift and unstable formatting	Long-context quality was limited more by output discipline than by reasoning depth.
v2	80.30	task-aware prompts plus BM25 retrieval	large immediate gain across short-output tasks	Explicit answer anchors and relevant demonstrations dominate early improvement.
v4	73.00	more aggressive reasoning and higher-variance decoding	degraded reliability despite plausible intermediate reasoning	Extra reasoning tokens can increase error surface under strict output contracts.
v5_fixed	80.55	conservative decoding, repaired validation, stronger retrieval packing	best logged checkpoint in the repository history	Stable prompting and normalization outperform more complex but unstable variants.
v8	79.60	later refinements and additional complexity	no clear improvement beyond the strong conservative regime	The benchmark punishes unnecessary system complexity.

recovery-path usage, invalid-output frequency, and task-level failure counts during dry runs. These diagnostics do not replace leaderboard evaluation, but they sharply reduce the probability that a reproduced submission fails for avoidable operational reasons such as context overflow, malformed outputs, or misaligned configuration files.

9. Per-Task Strategy Analysis

We devote one full section to task-by-task strategy differences because the benchmark is too heterogeneous for one uniform operating mode.

Tasks 1, 3, and 4 are also algorithmically solvable, and the repository reflects that in `hybrid_experiment/algo_solvers.py`. This matters because some subproblems are simply better handled by deterministic logic than by forcing everything through generation.

10. Failure Analysis and Mitigations

We keep failure analysis explicit because it shows how long-context systems fail operationally.

The failure analysis is consistent across tasks: format drift, context overflow, redundant demonstrations, ambiguous labels, interrupted batch execution, and brittle long-code outputs. The full matrix stays in the appendix (Table 17), but one point is worth keeping in the main text: output validation is part of the sys-

tem, not an afterthought.

11. Implementation and Reproducibility

Our current workspace contains two partially overlapping implementation strata. The compliant path is concentrated in `src/main.py`, `src/method.py`, and `src/llm_config.yaml`. The strategy-development history appears in the `hybrid_experiment/` directory, which contains multiple versioned inference scripts, utility tools, result folders, and planning notes.

In repository terms, the implementation boundary is clean. The official submission path is anchored in `src/main.py` for task-loop execution, `src/method.py` for prompt construction and retrieval, and `src/llm_config.yaml` for FlagScale deployment assumptions. The broader `hybrid_experiment/` directory preserves the development trail that motivated later design choices such as diversity control, task-adaptive voting, and stricter output repair.

In reproducibility terms, our most important alignment problem is configuration consistency. The tokenizer path, the prompt-budget logic, and the serving-side maximum context length must remain synchronized. If those settings drift apart, the example-selection logic can look correct in code while still failing in practice.

Table 7. Task-by-task strategy summary for Team Kima’s system.

Task	Core Skill	Primary Risk	Prompt Policy	Retrieval / Packing Policy	Post-Processing
Closest Integers	arithmetic transformation	prose drift around scalar answers	math_reasoning, integer-only output	compact demonstrations are sufficient	extract one valid integer
Count Nouns / Verbs	POS counting	counting drift and explanation	linguistic_analysis, integer-only output	BM25 is valuable because the pool is large	reject prose-like outputs
Collatz Transformation	elementwise symbolic mapping	list-format drift	math_reasoning with list examples	structure matters more than diversity	prefer last complete list
String Concatenation	exact string operation	whitespace or quotation artifacts	code_string, exact-match wording	a few exact demonstrations suffice	keep literal payload
Sadness Detection	binary social-text classification	keyword or emoji over-reliance	classification with holistic cues	some diversity helps reduce cue bias	normalize to legal labels
Same-Genre MNLI	pairwise textual reasoning	shallow lexical overlap heuristics	classification with genre reasoning	relevance and diversity both matter due to pool size	normalize to Y/N
Jeopardy QA	open answer generation	verbosity and near misses	open_generation, lowercase answer	coverage and diversity are important	keep one lower-case answer
Triton Kernel	long code generation	incomplete or brittle programs	code_generation, code-only output	token-density dominates; quality over quantity	preserve code verbatim, add missing imports only if needed

Table 8. Runtime environment snapshot extracted from the repository configuration.

Item	Configured value
Python environment	Conda environment with Python 3.11.11
Core framework	FlagScale serving plus vLLM backend
Model path	/root/autodl-tmp/qwen3-4b in src/llm_config.yaml
Maximum model length	max_model_len = 40000 with prefix caching enabled
Key libraries	PyTorch 2.6.0, vLLM 0.8.5, FlashAttention 2.8.3 wheel
Task execution scripts	path in setup script src/main.py, src/run_all.sh, src/package_results.sh

For direct reproduction, the shortest repository-aligned run path is also explicit in the codebase. We first launch the serving stack through FlagScale using src/llm_config.yaml, then execute src/run_all.sh to produce task-level jsonl files, and finally package the latest outputs through src/package_results.sh. This three-step path is intentionally simple because a competition report should make the operational route obvious rather than bury it in appendix prose.

Table 9. Configuration points that require explicit alignment.

Configuration Item	Alignment Requirement
Served model path	Must match the intended Qwen3-4B deployment path in the compliant track.
Maximum sequence length	The serving-side model length and the runtime prompt budget must agree.
Tokenizer path	Budget estimates should use the same tokenizer as the deployed model.
Prefix caching	Shared-prefix execution is most meaningful if the serving stack supports caching.
Voting policy	Repeated sampling should be enabled only where the task type actually benefits.
Packaging script	Final archive naming and task file collection must match the platform contract exactly.

12. Limitations and Scope

This report is a competition technical document, not a claim of architectural novelty. The work does not modify the Transformer architecture, does not argue that BM25 is universally optimal, and does not try to exhaust every ablation. The point is to document how the current repository turned long-context ICL into a stable submission pipeline under the benchmark rules.

We also distinguish the official submission code path from the broader internal development record. The submission path documents how we execute the Qwen3-4B + FlagScale pipeline required by the benchmark. The development record explains why certain prompt, retrieval, and validation choices survived system iteration and others did not.

13. Conclusion

This technical report documents how Team Kima improved long-context annotation quality through iterative prompt engineering, retrieval design, conservative decoding, and strong output validation. The version history around 80.55, 80.30, 79.78, 79.60, and 80.23 remains useful because it records which design choices looked stable or brittle in our local project trajectory.

The repository history points to a fairly plain conclusion. Stable output contracts mattered more than ornate prompting. Retrieval mattered most when the released example pool was too large to fit directly. Extra diversity and extra reasoning both had a narrow useful range and became harmful once they pushed the system away from local relevance or strict formatting. Low-temperature voting helped on some classification tasks and was mostly unnecessary elsewhere. Output extraction and normalization belonged in the core pipeline from the start.

A. Execution and Prompt Sketches

We keep both the official submission entry point and the broader development runner visible because they correspond to different parts of the same repository history.

B. Literature-to-Design Mapping

In this appendix, we make our literature-to-design mapping explicit rather than leaving it implicit in scattered citations. Our goal is not to claim theoretical novelty beyond the local system, but to show that our main design decisions are anchored in established findings about in-context learning, long-context efficiency, benchmark heterogeneity, and instruction quality.

Several points are worth emphasizing. First, our emphasis on prompt structure is a direct continuation of the ICL view advanced by Brown et al. rather than a cosmetic formatting preference. We treat the prompt as an executable task specification. Second, our reliance on retrieval is not merely a latency optimization. Under the long-example-pool regime documented in Table 4, retrieval is the only way for us to convert the benchmark’s released demonstrations into a useful in-context subset. Third, our selective use of voting follows the self-consistency literature only partially: our local experiments suggest that classification tasks benefit far more than generation-heavy tasks, so we adopt a narrower and more conservative policy than the generic literature might suggest.

The provided benchmark papers also help us explain why negative results matter. BIG-bench and SuperGLUE are both reminders that model quality is task-sensitive and that broad evaluation suites can punish one-size-fits-all prompting. In that sense, our local score regressions around v3 and v4 are not incidental engineering noise; they are informative examples of capability cliffs induced by apparently reasonable prompt choices.

C. Extended Task Notes

Our main paper already summarizes task-by-task strategy differences, but we document the reasoning behind those choices in slightly more detail here because several implementation decisions depend on task family.

C.1. Tasks 1–4: Structured Transformation Tasks

Tasks 1 through 4 benefit from strong output regularity. Their inputs may be natural-language descriptions or short symbolic fragments, but their outputs

have little tolerance for explanation text, paraphrase, or stylistic drift. This is why we repeatedly favor direct-answer prompting over chain-of-thought elaboration. For these tasks, demonstrations are most valuable when they illustrate exact output syntax and boundary conditions rather than when they provide long verbal rationales.

Our local repository further suggests that exact symbolic handling can outperform pure language-model inference on some of these tasks. We do not overstate that observation as a universal prescription, but we consider it technically important: long-context LLM pipelines are most credible when they admit that some subproblems are best solved by deterministic logic rather than by forcing every prediction through natural-language generation.

C.2. Tasks 5–7: Semantic Classification and QA

Tasks 5 through 7 place more weight on semantic alignment between the test input and retrieved demonstrations. In these tasks, our prompt must balance three constraints simultaneously: preserve a strict answer format, present enough semantically relevant examples, and avoid redundancy that wastes scarce context budget. This is precisely where our BM25+diversity packing story is strongest. Even if alternative embedding-based retrieval might eventually perform better, the lexical approach has the practical advantage of being deterministic and easy to audit.

Another reason for us to separate these tasks from the structured-transformation group is that uncertainty becomes intrinsic rather than merely syntactic. For binary classification tasks, voting can smooth over local instability. For Jeopardy-style short-answer generation, however, diversity in output can easily turn into near-miss proliferation. Our selective voting policy is therefore not just cost control; it is a task-theoretic distinction.

C.3. Task 8: Long-Code Generation

Task 8 is qualitatively different from the rest of the benchmark. The problem is not only semantic correctness, but also code completeness, import coverage, and output length. A prompt design that works well for short labels can fail badly once the target is thousands of characters of structured code. This is why we consistently prioritize code-only output, reduced prompt clutter, and strong length accounting for this task.

Our score history also suggests that adding more demonstrations is not always helpful here. Because

Table 10. Execution entry points used in Team Kima’s repository.

Official submission path	Internal development runner
<pre>cd FlagScale python run.py --config-path ../LongContext-ICL-Annotation/src \ --config-name llm_config action=run cd ../LongContext-ICL-Annotation python src/main.py --task_id 1 --tokenizer_path ./Qwen3-4B</pre>	<pre>cd LongContext-ICL-Annotation python hybrid_experiment/run_all_v8.py \ --output_dir ./hybrid_experiment/outputs_v8</pre>

Table 11. Compact prompt skeletons used by the main prompt families.

Classification-style skeleton	Code-generation skeleton
<pre>### Role You are a text classification expert. ### Task {task_definition} ### Instructions 1. Read the full input carefully. 2. Use all available cues. 3. Output only one valid label in <label>...</label>. ### Examples {retrieved_examples} ### Input {test_input} ### Output <label></pre>	<pre>### Role You are an expert Triton kernel programmer. ### Task {task_definition} ### Instructions 1. Write complete runnable code. 2. Include all required imports. 3. Do not emit explanation outside <label>...</label>. ### Examples {retrieved_examples} ### Input {test_input} ### Output <label></pre>

each example is token-dense, a small number of high-quality code exemplars can outperform a larger but noisier bundle. This is a concrete example of our broader claim that long context should be treated as a scarce allocation resource even when the nominal window is large.

D. Evaluation Protocol and Submission Checklist

To make our submission operationally useful, we spell out the run protocol that is only implicit in the main text.

Our insistence on this checklist is motivated by the competition format itself. In a conventional research paper, a strong qualitative output may be enough to support a claim. In an archive-based evaluation benchmark, the quality bar is different: a result is only as credible as the reproducible pipeline that generated it. This is another reason we spend so much effort on output normalization and configuration hygiene.

E. Additional Prompt Families

Our main paper gives only compact prompt skeletons. In this appendix, we record fuller variants so that the implementation details used in Team Kima’s system remain explicit.

F. Threats to Validity and Reporting Discipline

We intentionally distinguish the official submission pipeline from the broader internal development record. That choice introduces one limitation and resolves another.

The limitation is that not every logged score in our workspace is equivalent evidence for the official Qwen3-4B-only submission claim. We treat the broader development record as strategy-development evidence: it indicates which prompt structures, retrieval policies, and validation rules appeared robust in practice, but it is not a substitute for an independently re-executed official run under the exact benchmark constraints.

The advantage is that this separation prevents a common reporting failure in competition-style technical documents: collapsing all local experimentation into one undifferentiated number. By making the system boundary explicit, we make the report stricter, easier to audit, and ultimately more useful for future reproduction. This is consistent with the spirit of rigorous empirical reporting in machine learning, even when the document is framed as a competition technical report rather than as a conventional conference submission.

G. Archived Result Manifest

Our workspace history treats the submission trajectory itself as part of the design evidence. To preserve that audit trail in a cleaner form, we consolidate the result

Table 12. How the provided papers inform the present system design.

Reference	Core message	Concrete implication in our system
Brown et al. (2020)	Demonstrations in context can specify the task at inference time.	Treat examples as a controllable resource rather than as decorative context; shared-prefix ICL is the center of the system.
Tay et al. (2020a)	Long-sequence processing is constrained by computation and memory trade-offs.	Introduce tokenizer-aware budgeting and retrieval-based packing instead of naive full-context inclusion.
Tay et al. (2020b)	Long-context benchmarks are diverse and often stress different inductive biases.	Avoid a single universal prompt and instead specialize by task family and output contract.
Wang et al. (2019)	Benchmark suites with varied tasks reveal weaknesses hidden by single-task evaluation.	Interpret the eight FlagOS tasks as a heterogeneous suite requiring separate failure analysis.
Srivastava et al. (2022)	Broad benchmark collections expose unexpected capability cliffs and prompt sensitivity.	Preserve the version-by-version ablation narrative to show which “obvious” prompt additions actually degrade performance.
Wang et al. (2022)	Instruction structure strongly influences downstream model behavior.	Encode role, task, instructions, examples, input, and output anchor explicitly in every prompt family.
Wang et al. (2023)	Self-consistency can improve reasoning reliability under the right task conditions.	Use repeated low-temperature voting selectively for classification-style tasks rather than as a universal default.
Carbonell and Goldstein (1998)	Redundancy-aware reranking improves coverage under limited display budget.	Use diversity-aware example ordering to avoid filling context with near-duplicate demonstrations.

archives into one manifest. The intent is documentary rather than rhetorical: these entries record the observed iteration path and should be read together with Table 3.

H. Extended Ablation Matrix

We retain the more detailed ablation matrix here because it is useful for auditability even when the main paper only needs the distilled conclusions.

I. Extended Failure Matrix

We also preserve the repository-level failure matrix here because it explains why normalization, recovery, and strict output contracts remain central to the system.

Two empirical patterns stand out from this manifest. First, once our system entered the 80-point regime, subsequent revisions mostly moved within a narrow band rather than producing monotonic improvement. This supports our argument that the benchmark is sensitive to seemingly small prompt-budget and output-

validation choices. Second, the existence of several closely clustered scores strengthens our decision to foreground design principles rather than overfit the narrative to a single archive.

J. Per-Task Output Contracts

The final implementation layer of our system is an explicit task-by-task output contract. We record it separately because this contract is where prompt design, parsing, and evaluation criteria meet.

The practical value of Table 18 is that it converts abstract prompt-design principles into enforceable runtime behavior. For Tasks 1 through 7, the dominant risk is usually not that the model has no plausible answer, but that it wraps the answer in an invalid form. For Task 8, the opposite failure mode dominates: even semantically promising generations are useless if code completeness and executable structure break.

This is why we consider output contracts part of the method rather than just part of evaluation cleanup. In long-context annotation systems, the final metric is

Table 13. Operational checklist for a compliant-style submission run.

Stage	Verification item
Model serving	Confirm that the served model path points to Qwen3-4B and that FlagScale/vLLM starts successfully.
Tokenizer alignment	Ensure that prompt budgeting uses the same tokenizer family as the served model.
Task loading	Verify that all eight benchmark task files are loaded and that example pools are parsed correctly.
Prompt selection	Confirm that each task is routed to the intended prompt family rather than to a generic fallback.
Retrieval budget	Check that retrieved demonstrations are packed under the actual token budget with safety margin.
Inference policy	Use low-temperature decoding by default; enable voting only for tasks where the policy explicitly calls for it.
Output validation	Normalize outputs into legal task labels, exact strings, or code blocks before writing jsonl.
Archive packaging	Confirm that per-task result files are present and zipped exactly as expected by the platform.
Audit trail	Save configuration files, runtime logs, and result hashes for post-hoc verification.

Table 14. Extended prompt-family variants used for symbolic and open-generation tasks.

Math / symbolic prompt family	Open-generation prompt family
<pre> ### Role You are an expert in exact symbolic reasoning. ### Task Solve the task exactly according to the definition. ### Instructions 1. Follow the pattern shown in the examples. 2. Do not explain your reasoning. 3. Output only the final answer in <label>...</label>. ### Examples {retrieved_examples} ### Input {test_input} ### Output <label> </pre>	<pre> ### Role You are a concise question answering system. ### Task Provide the most precise short answer. ### Instructions 1. Use the examples to infer the expected answer style. 2. Prefer the shortest exact answer that fits the evidence. 3. Do not repeat the question or add explanation. 4. Output only one lowercase answer in <label>...</label>. ### Examples {retrieved_examples} ### Input {test_input} ### Output <label> </pre>

applied to structured artifacts, not to latent intent. A prompt policy, parser, and normalizer that agree on the same contract are therefore a necessary condition for reliable submission quality.

because it shows why we reject one-size-fits-all prompting: answer anchoring is nearly universal, retrieval matters most for large-pool semantic tasks, and code repair is only meaningful for the Triton setting.

K. Task Landscape Visualization

We also summarize the benchmark geometry visually because the long-context challenge is easier to read from the joint distribution of example-pool size and output length than from isolated statistics. The figure below makes one asymmetry especially clear: Task 8 is small in example count but extremely heavy in output length, while Tasks 2, 6, and 7 are large-pool retrieval tasks with short outputs.

L. Control Mechanism Matrix

Finally, we expose the relative importance of each control mechanism across tasks. This summary is useful

Table 15. Archived result files and recorded platform outcomes from Team Kima’s development history.

Archive / run label	Recorded timestamp	Score	Interpretation in this report
all_jsonl_files.zip	2026-05-11 19:29:03	80.30	Early strong archive-level milestone in the broader local run history.
hybrid_results-yyds.zip	2026-05-16 18:08:30	77.20	Earlier exploratory hybrid attempt before the strongest prompt and validation refinements stabilized.
hybrid_results.zip	2026-05-16 22:36:33	80.23	Competitive exploratory checkpoint showing that disciplined prompting already outperformed naive baselines.
hybrid_results.zip	2026-05-16 22:49:04	80.15	Near-duplicate run indicating mild sensitivity to configuration or batch-level randomness.
hybrid_results.zip	2026-05-16 23:58:00	80.18	Stable high-score regime before the v5_fixed peak.
hybrid_results_v5_fixed.zip	2026-05-17 01:31:40	80.55	Best logged exploratory score; treated as the most important development checkpoint in the narrative.
hybrid_results_v6.zip	2026-05-17 16:28:12	79.78	Follow-up run suggesting that later refinements did not strictly dominate v5_fixed.
hybrid_results_v8.zip	2026-05-17 21:22:54	79.60	Later exploratory archive reinforcing the conclusion that more complexity did not guarantee better results.

Table 16. Ablation-style lessons distilled from Team Kima’s local version progression.

Ablation Axis	Compared Settings	Main Observation	Interpretation
Prompt granularity	generic prompt vs. task-aware prompt families	task-aware prompts reduce format drift substantially	Output contracts matter as much as semantic instructions.
Example selection	sequential truncation vs. BM25 retrieval	retrieval helps most on Tasks 2, 6, and 7 with very large example pools	Demonstration choice matters more when brute-force inclusion is impossible.
Diversity control	relevance only vs. relevance plus diversity	moderate diversity helps, but aggressive diversity hurts	Coverage is useful only when it does not destroy local relevance.
Voting policy	single sample vs. repeated low-temperature voting	classification tasks benefit more than generation tasks	Stability and plurality are task dependent, not universal.
Reasoning style	direct answer vs. CoT	CoT often hurts constrained classification tasks	Extra reasoning tokens can be an error source rather than a benefit.
Decoding schedule	fixed low temperature vs. annealed temperature	fixed low temperature gives stronger consensus	High-variance sampling is poorly matched to rigid output tasks.

Table 17. Failure taxonomy observed during Team Kima’s development.

Failure Mode	Typical Source	Mitigation	Residual Risk
Format drift	prose before or after the answer	explicit answer anchor plus multi-stage extraction	long code outputs can still challenge parsers
Context overflow	too many long demonstrations	tokenizer-aware packing and run-time length guards	extreme cases can force aggressive truncation
Demonstration redundancy	top lexical hits cluster around one pattern	diversity-aware reordering	output-pattern diversity may miss deeper semantic redundancy
Label ambiguity	semantically close classes	task-specific prompts and optional voting	intrinsically uncertain samples remain unstable
Empty predictions	API timeouts, rate limits, or interrupted runs in the exploratory track	post-hoc repair from earlier checkpoints, stricter batch control	repair logic depends on prior saved outputs
Code brittleness	missing imports or partial programs in Task 8	code-specific prompt family and light normalization	execution correctness remains harder than string normalization

Table 18. Per-task output contracts enforced by Team Kima’s pipeline.

Task	Legal output form	Normalization rule	Failure if the contract is violated
Closest Integers	one integer	keep the final valid scalar inside <label> tags	extra prose or multiple numbers can corrupt exact-match parsing
Count Nouns / Verbs	one integer count	strip wrappers and reject prose-like answers	explanations often hide the intended count
Collatz Transformation	one integer list	preserve the last complete valid list	partial sequences fail exact comparison
String Concatenation	one exact string	keep literal content including spacing and punctuation	tiny formatting drift becomes a wrong answer
Sadness Detection	one legal class label	map variants into the official binary label set	informal wording can become invalid class output
Same-Genre MNLI	Y or N	collapse synonyms into the required binary form	semantically correct prose can still be operationally invalid
Jeopardy QA	one short lowercase answer	lowercase and keep a single concise span	near-miss verbosity harms exact answer matching
Triton Kernel	one complete code block	preserve code verbatim and add only minimal repairs if needed	partial or mixed natural-language output fails executable evaluation

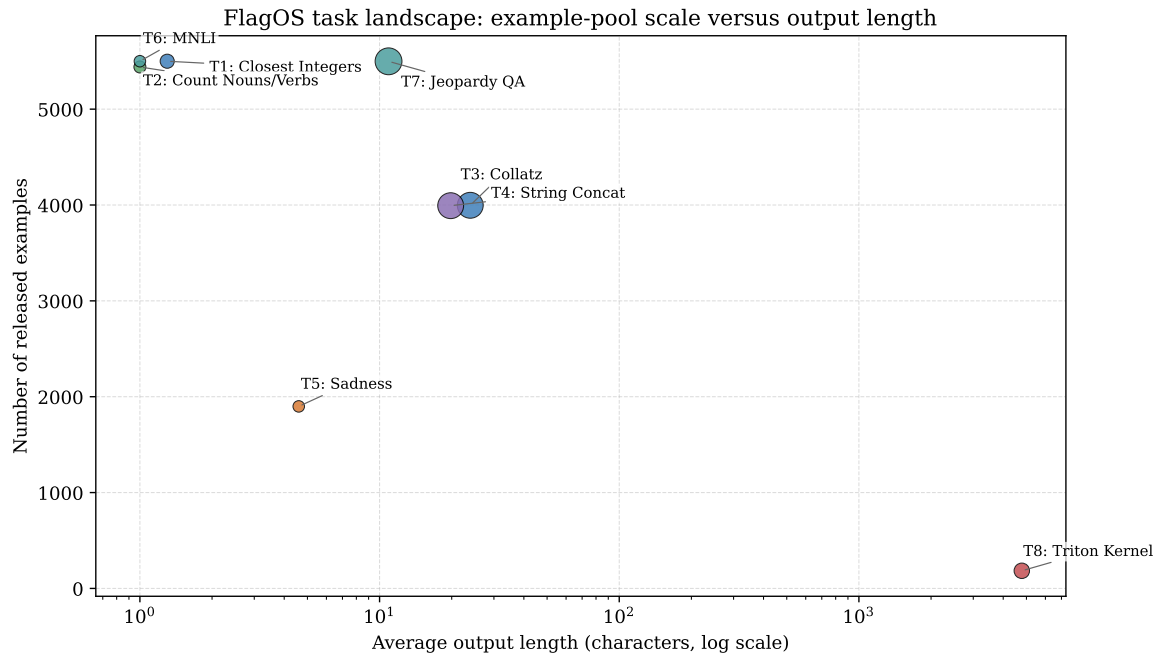


Figure 3. Task landscape of the FlagOS benchmark. The benchmark mixes large-pool retrieval-oriented tasks with very short outputs and a code-generation task with dramatically longer outputs.

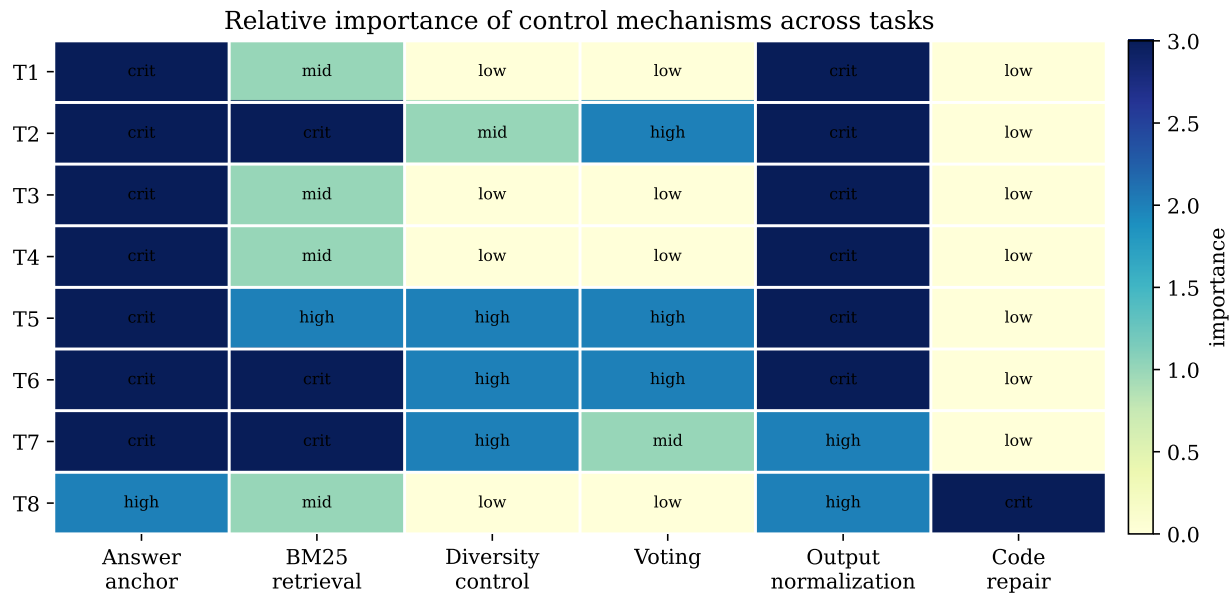


Figure 4. Relative importance of different control mechanisms across tasks in Team Kima’s pipeline. The matrix clarifies why prompt anchors, retrieval, voting, and post-processing should be activated selectively rather than uniformly.

References

- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. 2020. Language Models are Few-Shot Learners. arXiv preprint arXiv:2005.14165.
- Carbonell, J. and Goldstein, J. 1998. The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries. In *Proceedings of SIGIR 1998*.
- FlagOS. 2026a. Long-Context Automatic Data Annotation Challenge Specification. Competition documentation released with the FlagOS benchmark.
- FlagOS. 2026b. Q&A for the Long-Context Automatic Data Annotation Challenge. Official organizer clarification document for model, data, and framework constraints.
- Pilault, J., Garcia, X., Bražinskas, A., and Firat, O. 2023. Interactive-Chain-Prompting: Ambiguity Resolution for Crosslingual Conditional Generation with Interaction. arXiv preprint arXiv:2301.10309.
- Srivastava, A., Rastogi, A., Rao, A., Shoeb, A. A. M., Abid, A., Fisch, A., Brown, A. R., Santoro, A., Gupta, A., Garriga-Alonso, A., Kluska Agyei, A., et al. 2022. Beyond the Imitation Game: Quantifying and Extrapolating the Capabilities of Language Models. *Transactions on Machine Learning Research*; originally released as arXiv:2206.04615.
- Tay, Y., Dehghani, M., Bahri, D., and Metzler, D. 2020a. Efficient Transformers: A Survey. arXiv preprint arXiv:2009.06732.
- Tay, Y., Dehghani, M., Abnar, S., Shen, Y., Bahri, D., Pham, H., Rao, J., Liu, M.-W., Yang, F., Ruder, S., and Metzler, D. 2020b. Long Range Arena: A Benchmark for Efficient Transformers. arXiv preprint arXiv:2011.04006.
- Wang, A., Pruksachatkun, Y., Nangia, N., Singh, A., Michael, J., Hill, F., Levy, O., and Bowman, S. R. 2019. SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems. *Advances in Neural Information Processing Systems*; originally released as arXiv:1905.00537.
- Wang, Y., Kordi, Y., Mishra, S., Liu, A., Smith, N. A., Khashabi, D., and Hajishirzi, H. 2022. Self-Instruct: Aligning Language Model With Self Generated Instructions. arXiv preprint arXiv:2212.10560.
- Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations*.